Concurrency Control for Shared Memory in Multi-Agent Systems

Ashwattha Phatak
North Carolina State University
Raleigh, NC, USA
aphatak@ncsu.edu

Ayush Gala
North Carolina State University
Raleigh, NC, USA
agala@ncsu.edu

ABSTRACT

Modern multi-agent systems built on large language models increasingly share a common vector database as a persistent memory layer. Classical concurrency control mechanisms protect row-level or key-level identity, leaving a gap: two agents writing semantically equivalent content with different wording represent a conflict that no existing lock manager detects. We present the **Distributed Semantic Conflict Controller (DSCC)**, a distributed lock manager that uses cosine similarity over embedding vectors as its conflict predicate. DSCC admits a write only when the incoming embedding lies below a configurable similarity threshold θ relative to all active locks; otherwise the request blocks in a per-lock FIFO queue. Lock state is replicated across a five-node Raft cluster for fault tolerance, and a leader-aware C++ proxy shields clients from leader changes. The central contribution is the **ActiveLockTable** design: a per-lock waiter queue with per-waiter condition variables that eliminates thundering-herd contention while preserving FIFO fairness through a cascading grant rebalancer. We evaluate three embedding models across 13 benchmark scenarios. Only `qwen3-embedding:0.6b` at θ=0.75 achieves a perfect serialization score of 1.000 with zero paraphrase violations, at a write throughput of 32 RPS with a median latency of 10 ms and P95 of 68 ms.

KEYWORDS

distributed concurrency control, semantic locking, multi-agent systems, vector databases, Raft consensus, embedding models

1 INTRODUCTION

The emergence of multi-agent systems built on large language models has introduced a new class of shared-memory infrastructure: *vector databases*. Collections such as Qdrant [9] store dense embedding vectors alongside natural-language payloads and serve as the long-term memory for agent pipelines. When multiple agents write to the same collection concurrently, a correctness question arises: how does the system prevent two agents from independently persisting the same piece of knowledge expressed in different words?

Traditional distributed locking [1, 2] guards against concurrent access to the same *key*. An agent writing *"prioritize passive cooling and low-carbon materials"* and another writing *"evaluate daylight access and sustainable envelope decisions"* would each receive their own key in a vector store and would not conflict under any key-based scheme. Yet both sentences encode the same architectural design intent; allowing them to persist concurrently introduces a *semantic race condition* — duplicated, inconsistent knowledge in the shared store.

We define a semantic race condition as the concurrent execution of two write operations whose embedding vectors satisfy $\cos(\mathbf{v}_A, \mathbf{v}_B) \geq \theta$ for some deployment-calibrated threshold θ. The **DSCC** (Distributed Semantic Conflict Controller) prevents such races by admitting a write only after verifying that no active lock's centroid is within cosine distance $1-\theta$ of the incoming embedding.

This paper makes the following contributions:

(1) We formalize the notion of a *semantic race condition* and define a cosine-similarity-based conflict predicate for distributed lock management (§3).
(2) We present the **ActiveLockTable**, a concurrent in-memory data structure with per-lock FIFO waiter queues and per-waiter condition variables, together with a cascading grant rebalancer that preserves fairness after each lock release (§6).
(3) We describe a two-phase admission protocol that integrates the lock table with a five-node Raft consensus layer, including a ghost-lock prevention mechanism for failed Raft proposals (§5).
(4) We conduct an embedding model study across three models and three θ values, demonstrating that the choice of embedding model is an architectural decision—not an implementation detail—and identifying the recommended operating point (§7).
(5) We evaluate the full system against 13 benchmark scenarios and under sustained load testing, reporting latency, throughput, fairness, and resource utilization (§8).

2 BACKGROUND AND RELATED WORK

2.1 Distributed Locking

Chubby [1] introduced coarse-grained distributed locking using a replicated lock service built on Paxos. Clients acquire advisory locks on named nodes in a file system hierarchy; lock identity is purely lexicographic. ZooKeeper [2] generalized this with a hierarchical namespace and ephemeral znodes, which has been used to implement distributed mutexes, leader election, and barriers. Redlock [3] extends Redis to distributed lock acquisition via quorum across independent instances. None of these mechanisms consider the *content* of the protected resource; they serialize access to a name, not to a meaning.

2.2 Concurrency Control in Databases

Two-phase locking [5] and optimistic concurrency control [6] operate on rows or pages identified by primary keys or physical addresses. Multi-version concurrency control [7] extends this

to snapshot reads. Semantic concurrency control [8] introduced the idea that two operations may be compatible if their effects commute, but this was defined over database operations rather than natural-language content.

2.3 Vector Databases and Approximate Nearest-Neighbor Search

Vector databases such as Qdrant [9], Weaviate, Pinecone, and pgvector extend relational and document stores with dense vector indexes for approximate nearest-neighbor (ANN) retrieval. Efficient ANN algorithms [10, 11] enable sub-millisecond query at scale. These systems expose no concurrency abstraction above the collection level; concurrent writers producing semantically equivalent vectors are indistinguishable from writers producing unrelated vectors.

2.4 Multi-Agent Memory and Coordination

Concurrent agent frameworks such as LangGraph [12] and AutoGen [13] provide orchestration primitives for LLM agents but delegate memory consistency to the underlying store. Generative Agents [14] demonstrated that agents benefit from a shared memory of observations, but the multi-agent write conflict problem was not addressed. ReAct [15] and similar agent architectures assume a single-agent context. DSCC targets the gap: a lock manager whose conflict predicate operates on the semantic space of agent payloads.

2.5 Raft Consensus

Raft [4] provides understandable distributed consensus with leader election and log replication. Our implementation closely follows the original description, with sequential vote collection (a known simplification) and an in-memory log (no snapshots or durable state). The consensus layer in DSCC is responsible for making lock acquisition and release decisions durable across the cluster.

3 PROBLEM FORMULATION

3.1 System Model

We consider a set of n agents $\{a_1, \ldots, a_n\}$ writing to a shared vector database $\mathcal{D}$. Each write request carries a natural-language payload $s \in \Sigma^*$ that the embedding service maps to a vector $\mathbf{v} = \text{Embed}(s) \in \mathbb{R}^d$.

3.2 Semantic Conflict

Two requests r_A and r_B with embeddings $\mathbf{v}_A$ and $\mathbf{v}_B$ are *semantically conflicting* if:

$$\text{sim}(\mathbf{v}_A, \mathbf{v}_B) = \frac{\mathbf{v}_A \cdot \mathbf{v}_B}{\|\mathbf{v}_A\| \, \|\mathbf{v}_B\|} \geq \theta \tag{1}$$

for a threshold $\theta \in (0, 1]$. The threshold is not an absolute semantic boundary; it is calibrated relative to a specific embedding model and workload (see §7).

3.3 Semantic Race Condition

A *semantic race condition* occurs when two conflicting requests execute concurrently: their writes both commit to $\mathcal{D}$ without any ordering guarantee. The result is duplicated or inconsistent knowledge in the shared store. Classical isolation levels — read committed, repeatable read, serializable — do not apply because the conflict is defined over the payload semantics, not over row identity.

3.4 Semantic Serializability

We define a schedule as *semantically serializable* if, for every pair of conflicting requests (r_A, r_B), one completes before the other begins its Qdrant write. The **serialization score** of a benchmark run is the fraction of conflicting pairs that were correctly serialized:

$$\text{serialization\_score} = 1 - \frac{\text{violations}}{|\text{conflict pairs}|}$$

A score of 1.000 means zero semantic race conditions were observed.

3.5 Design Goals

(1) **Correctness**: serialize all semantically conflicting writes.
(2) **Parallelism**: allow unrelated writes to proceed concurrently.
(3) **Fairness**: blocked requests are served in FIFO order per conflict group.
(4) **Fault tolerance**: lock state survives the loss of up to two nodes in a five-node cluster.

4 SYSTEM ARCHITECTURE

Figure 1 shows the end-to-end architecture. The system has four runtime layers: (1) the **embedding service** (Ollama), (2) the **leader-aware proxy** (`dscc-proxy`), (3) the **five-node DSCC cluster**, and (4) the **Qdrant vector database**. All inter-service communication uses gRPC.

4.1 Embedding Service

Before any lock operation, the agent's payload is vectorized by Ollama. The resulting embedding vector is attached to the gRPC `AcquireRequest` alongside the similarity threshold θ. The lock table itself never interprets natural language; it only compares floating-point vectors. This clean separation means the embedding model can be swapped without changing the locking logic (see §7).

4.2 Leader-Aware Proxy

`dscc-proxy` is a C++ gRPC server that shields clients from leader changes. A background `LeaderPollLoop` issues `GetLeader` RPCs every 100 ms to maintain a cached view of the current leader address. Client-facing requests are forwarded via `ForwardWithLeaderRetry`: if the first attempt fails or returns a "not leader" error, the proxy refreshes its leader cache and retries, up to three attempts with a 35-second per-attempt deadline. gRPC channels to DSCC nodes are lazily created and cached; the proxy does not open a connection to a node until the first request destined for it.

4.3 DSCC Cluster

Each of the five DSCC nodes runs an identical binary that wires together a `RaftNode`, an `ActiveLockTable`, and a `LockServiceImpl` (the gRPC service handler). Only the leader accepts `AcquireGuard` and `ReleaseGuard` requests. Followers maintain a replica of the

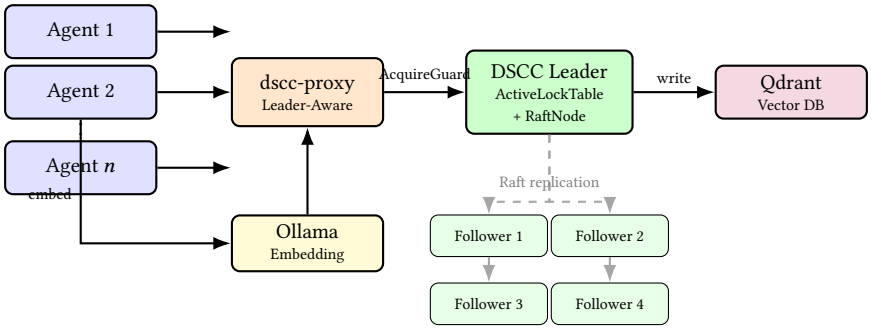


Figure 1: DSCC system architecture. Agents embed their payload via Ollama, then send `AcquireGuard` through the leader-aware proxy. The proxy forwards to the current Raft leader, which runs admission through the ActiveLockTable and replicates the decision to four followers (dashed) before returning. The committed write then proceeds to Qdrant.

lock state through Raft log application and are ready to become leader if the current leader fails.

The cluster is deployed on Docker Compose. Each node is allocated its own container, and Qdrant runs in a sixth container. Container resource limits for CPU and memory are configured per-node to emulate heterogeneous hardware.

5 RAFT CONSENSUS LAYER

5.1 Configuration and Threading

The Raft cluster has N=5 nodes, requiring a quorum of 3 and tolerating 2 simultaneous failures. Each node runs three background threads: a `HeartbeatLoop` (75 ms period) that sends `AppendEntries` RPCs to all peers; an `ElectionTimerLoop` (10 ms poll, 600–1000 ms randomized deadline) that triggers candidate campaigns; and an `ApplyLoop` that delivers committed log entries to the `on_commit_` callback.

5.2 Two-Phase Admission

A single `AcquireGuard` call on the leader traverses three phases:

(1) **Admission (Phase 1).** `wait_for_admission()` blocks on the ActiveLockTable until no active lock conflicts with the incoming embedding. On clearance, it inserts a *pending* slot — a reservation that participates in conflict detection before Raft commits (see §5.3).
(2) **Consensus (Phase 2).** The leader calls `Propose(ACQUIRE)`, which appends the entry to the Raft log and waits for a quorum `AppendEntries` acknowledgement via `WaitUntilApplied`.
(3) **Promotion.** The `on_commit_` callback fires on every node. On the leader, `apply_acquire()` finds the pending entry and clears the `pending` flag, promoting it to a real lock. On followers, no pending slot exists, so `apply_acquire()` inserts a fresh real lock directly.

The agent is then free to execute its Qdrant write. After the write completes, `ReleaseGuard` issues `Propose(RELEASE)` and the lock is removed from the table on all nodes through the same apply callback path.

5.3 Pending Lock Semantics

The pending flag solves a race between the end of Phase 1 and the completion of Phase 2. Without a reservation, a second agent could observe the lock table between the moment the first agent clears the conflict check and the moment the Raft commit makes its lock visible. The pending slot closes this window: it participates in `find_conflict_locked` immediately after `wait_for_admission` returns, before any Raft round-trip.

5.4 Ghost Lock Prevention

If `Propose(ACQUIRE)` times out or the leader loses leadership during Phase 2, the pending slot must be cleaned up. The leader calls `remove_pending(agent_id)`, which removes the pending entry and rebalances any waiters queued behind it — but only if the entry is still pending. If the Raft commit had already promoted it, a compensating `Propose(RELEASE)` is submitted instead (*AppendLocalEntry* path). This ensures that ghost locks — pending entries that were never promoted but also never removed — cannot accumulate and block the system indefinitely.

5.5 Sequential Vote Collection — A Known Limitation

Vote requests during leader election are currently issued sequentially rather than in parallel, giving $O(N \times \text{rpc\_timeout})$ election time in the worst case. On a healthy local Docker network this is acceptable, but on a high-latency WAN it would significantly delay recovery. Additionally, the implementation lacks a *pre-vote* phase [4]: a temporarily partitioned node can increment its term and disrupt the cluster on reconnect. Both are identified as open limitations in §9.

6 THE ACTIVELOCKTABLE

The ActiveLockTable is the central contribution of this paper. It is an in-memory concurrent data structure that answers two questions for every incoming request: *"Does this request conflict with any active lock?"* and, if so, *"Who should this request wait behind, and in what position?"*

6.1 Early Design and Its Failure Mode

The initial implementation used the simplest possible representation: a flat `vector<SemanticLock>` protected by a single mutex and a single global `condition_variable`:

Listing 1: Early single-CV design

```
class ActiveLockTable {
    std::vector<SemanticLock> active_;
    mutable std::mutex mu_;
    std::condition_variable cv_;  // shared by all waiters
};
```

When any lock was released, `cv_.notify_all()` woke *every* blocked thread. Each woken thread then re-evaluated the conflict predicate under the mutex. In the thundering-herd test case (13 agents competing for a single semantic region), this caused a burst of 12 simultaneous wakeups, 11 of which immediately went back to sleep after re-discovering a conflict. Under contention, the mutex became a serialization point for all woken threads, and the lock table throughput degraded nearly linearly with the number of waiting agents.

Additionally, the flat vector required $O(n)$ scans for both conflict checks and lock removal, and there was no concept of per-lock waiter ordering — a newly woken thread could jump ahead of a thread that had been waiting longer, violating FIFO fairness.

6.2 Current Data Model

The production design replaces the vector and shared CV with two structural changes:

Listing 2: Current per-waiter CV design

```
struct WaitQueueEntry {
    std::string waiting_agent_id;
    std::vector<float> embedding;
    float theta;
    std::shared_ptr<std::condition_variable> cv;
    bool ready{false};
    bool granted{false};
    int queue_hops{0};
};

struct SemanticLock {
    std::string agent_id;
    std::vector<float> centroid;
    float threshold;
    bool pending;
    std::deque<std::shared_ptr<WaitQueueEntry>> waiters;
};

class ActiveLockTable {
    std::unordered_map<std::string, SemanticLock> active_;
    mutable std::mutex mu_;
    // No shared CV — each WaitQueueEntry has its own.
};
```

Key design choices:

Per-lock FIFO deque. Each `SemanticLock` owns a deque of `WaitQueueEntry` objects, not a flat list of all blocked threads. When a lock is released, only the waiters attached to that lock are processed.

Per-waiter condition variable. Every `WaitQueueEntry` has its own `condition_variable`. The rebalancer notifies only the CVs of threads it decides to grant, not all threads. This eliminates the thundering herd entirely.

Two wakeup reasons. The `ready` flag gates the CV wait; `granted` distinguishes a grant handoff (the rebalancer already inserted a pending slot and the thread must not re-run the conflict check) from a spurious wakeup (the thread re-enters the while loop).

$O(1)$ **lookup.** The `unordered_map` keyed by `agent_id` gives constant-time lock insertion, removal, and promotion from pending to real.

6.3 Conflict Detection: Strongest-Blocker Selection

`find_conflict_locked` scans all entries in `active_` and computes cosine similarity between the incoming embedding and each lock's centroid (Equation 1). If multiple locks conflict, the waiter is attached to the one with the *highest* similarity — the strongest blocker:

Listing 3: Strongest-blocker selection

```
ConflictTrace best;
for (auto& [agent_id, entry] : active_) {
    float sim = cosine_similarity(embedding, entry.centroid);
    if (sim >= threshold &&
        (best.lock == nullptr || sim >= best.similarity)) {
        best.lock = &entry;
        best.similarity = sim;
    }
}
return best;
```

The rationale is that a waiter attached to its most semantically similar blocker will experience the minimal number of requeue hops: when the strongest blocker releases, the waiter is most likely to pass the re-evaluation. Attaching to a weaker blocker that releases earlier would require the waiter to be requeued behind the stronger one, adding a hop and additional latency.

Threshold asymmetry. The conflict check always uses the *incoming* request's θ, not the active lock's. A conservative client with low θ can therefore be blocked by an active lock that would not have considered the new request a conflict at its own threshold.

6.4 The Rebalancer: `rebalance_waiters_locked`

When a lock is released, its waiter deque is detached and passed to `rebalance_waiters_locked`. The rebalancer processes waiters front-to-back (FIFO order):

Listing 4: Rebalancer core loop

```
while (!waiters.empty()) {
    auto waiter = waiters.front();
    waiters.pop_front();

    ConflictTrace conflict =
        find_conflict_locked(waiter->embedding, waiter->theta);

    if (conflict.lock != nullptr) {
        // Still blocked — requeue behind new strongest blocker
        ++waiter->queue_hops;
        conflict.lock->waiters.push_back(waiter);
        continue;
    }

    // Granted — insert pending slot and notify after mu_ released
    insert_pending_locked(waiter->waiting_agent_id,
                          waiter->embedding, waiter->theta);
    waiter->granted = true;
    waiter->ready  = true;
```

Table 1: Evaluated embedding models

Model	Dims	Emb. p50	Emb. p99
`all-minilm:latest`	384	10 ms	324 ms
`bge-m3:latest`	1024	111 ms	1774 ms
`qwen3-embedding:0.6b`	1024	133 ms	1517 ms

```
    granted_waiters.push_back(std::move(waiter));
}
```

All mutations happen under `mu_`. CV notifications fire *after* `mu_` is released, preventing a waking thread from contending with ongoing rebalancing.

6.5 Cascading Grant Effect

The most subtle behavior arises when multiple waiters can be granted in a single rebalance pass. Consider: active locks X (agent A) and Y (agent B), with X's waiters $[W_1, W_2]$. Agent A releases X. The rebalancer processes the detached queue $[W_1, W_2]$:

(1) W_1 is evaluated against $\{Y\}$. No conflict — W_1 is granted and inserted into `active_` as pending. Now `active_` $= \{Y, W_1\}$.
(2) W_2 is evaluated against $\{Y, W_1\}$. If W_2 conflicts with W_1, W_2 is requeued behind W_1, with `queue_hops` incremented. If W_2 does not conflict with either, W_2 is also granted.

The key insight is that granting W_1 can *block* W_2 in the same rebalance pass. Because the rebalancer processes front-to-back, earlier waiters get priority — FIFO fairness is preserved within the conflict group. The `queue_hops` counter tracks how many times a waiter has been bounced across lock queues; Figure 9b shows this effect in the Queue-Hopping benchmark case.

6.6 Notification Safety

After all rebalancing completes, `mu_` is released and each granted waiter's `cv->notify_all()` is called. The waking thread finds `granted` set to `true`, skips the conflict re-check, and proceeds directly to `Propose(ACQUIRE)`. The `WaitQueueEntry` shared pointer is kept alive by both the sleeping thread and the granted-waiters list; the rebalancer's copy is dropped after notification.

7 EMBEDDING MODEL AS CONFLICT PREDICATE

7.1 Why Model Choice Is Architectural

The ActiveLockTable receives only floating-point vectors and a scalar threshold; it contains no NLP logic. Therefore, the embedding model controls the *geometry of the lock space*: which payloads are considered conflicting, which can run in parallel, and how the threshold should be interpreted. Two agents writing paraphrases of the same intent will block each other only if the model places their embeddings above θ. If the model fails to cluster paraphrases tightly, the lock manager faithfully allows them to run concurrently — producing a semantic race condition that no amount of consensus correctness can fix.

We evaluated three models (Table 1) across three threshold values ($\theta \in \{0.55, 0.75, 0.95\}$) and 13 benchmark scenarios.

Table 2: Paraphrase Gauntlet at $\theta = 0.75$

Model	Serialization Score	Violations
`all-minilm`	0.500	6
`bge-m3`	0.750	3
`qwen3`	**1.000**	**0**

7.2 Paraphrase Gauntlet Results

The Paraphrase Gauntlet (Case 11) presents 12 pairs of semantically equivalent sentences drawn from distinct domains — sustainability design, legal reasoning, financial analysis — to test whether the system correctly serializes each pair. Table 2 shows the serialization score and violation count at $\theta = 0.75$.
Only `qwen3-embedding:0.6b` at θ=0.75 achieves a perfect serialization score.

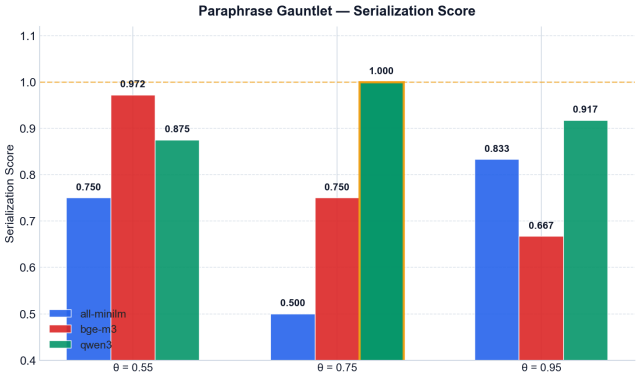


Figure 2: Serialization scores across all three models and three threshold values on the Paraphrase Gauntlet. `qwen3` at θ=0.75 is the only combination achieving a score of 1.000.

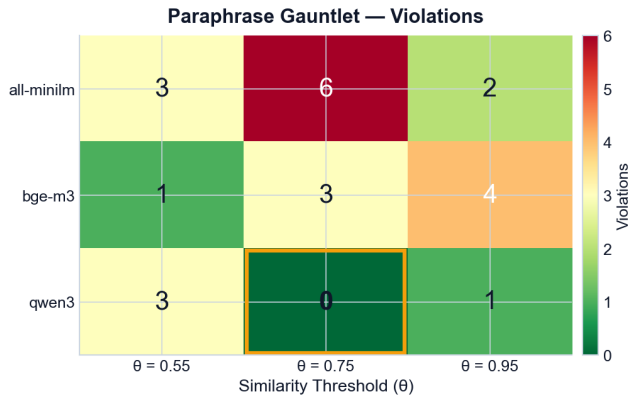


Figure 3: Violation heatmap across models and threshold values. Darker cells indicate more paraphrase race conditions observed. `all-minilm` produces the most violations at every tested threshold.

Figure 2 shows the full serialization score surface. The non-monotonic behavior of `all-minilm` across thresholds illustrates why lowering θ does not compensate for a weak paraphrase model: a lower threshold increases blocking, but the wrong operations are being blocked. Figure 3 confirms that `all-minilm` accumulates the most violations at every threshold.

7.3 Precision, Recall, and the False-Negative Risk

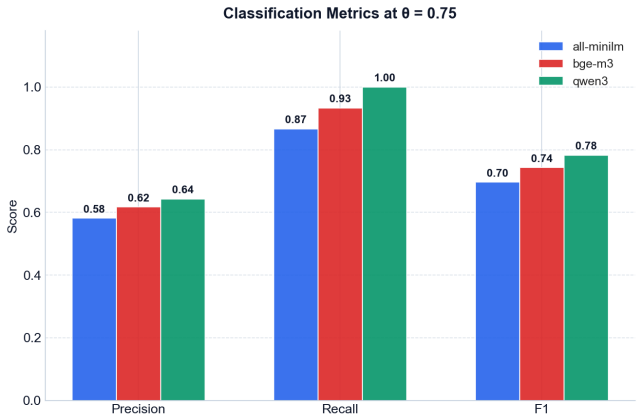


Figure 4: Precision, Recall, and F1 across models and threshold values. Higher recall is critical; a false negative is a semantic race condition.

We frame conflict detection as binary classification: a true positive is a conflicting pair that was serialized; a false negative is a conflicting pair that was allowed to race. Figure 4 shows precision, recall, and F1. A false negative is strictly more dangerous than a false positive (excessive blocking): missed paraphrase conflicts corrupt the shared memory store, while over-blocking merely reduces throughput. The evaluation therefore prioritizes recall over precision.

7.4 Confusion Matrices at $\theta = 0.75$

Figure 5 provides per-model confusion matrices at θ=0.75. The matrices confirm that `qwen3`'s advantage comes from recall: it produces zero false negatives while maintaining comparable precision to the other models at this threshold.

7.5 Threshold Sensitivity

Threshold calibration is model-specific. At θ=0.55, both `bge-m3` and `qwen3` exhibit severe cross-domain false-positive blocking in Case 12 (Cross-Domain Flood), with distinct parallelism collapsing to 0.000 and operation P95 exceeding 5500 ms. At θ=0.95, paraphrase recall degrades for all models as pairs drift below the threshold. The calibrated operating point — `qwen3` at θ=0.75 — sits in the stable region: paraphrases serialize, unrelated domains proceed in parallel.

Table 3: Benchmark scenario catalog

Case	Name	Purpose
C1	The Thundering Herd	13 agents, one semantic region; tests per-waiter CV design
C2	Semantic Interleaving	Alternating related/unrelated writes; tests parallelism
C3	Read Starvation Trap	Reader-writer priority under write pressure
C4	Permissive Sieve	Low-θ blocking; tests false-positive rate
C5	Strict Sieve	High-θ blocking; tests false-negative rate
C6	Ghost Client	Proxy reconnect after leader change; tests ghost lock prevention
C7	Almost Collision	Near-threshold pairs; boundary precision test
C8	Queue Hopping	Multiple conflict groups; tests queue_hops counter
C9	Mixed Stagger	Time-staggered arrivals; tests fairness
C10	100-Read Stampede	Bulk reads with concurrent writes; throughput stress
C11	Paraphrase Gauntlet	12 paraphrase pairs; primary correctness test
C12	Cross-Domain Flood	Unrelated domains; tests false-positive parallelism
C13	Write Pressure Ratchet	Increasing write rate; latency under escalating load

7.6 Embedding Overhead

Figure 6 shows the embedding latency distribution. The p50 overhead of 133 ms for `qwen3` is bounded and predictable under steady load. In contended scenarios the lock wait component dominates end-to-end latency by a wide margin; the embedding overhead is a fixed per-request cost that does not scale with queue depth.

8 EVALUATION

8.1 Benchmark Suite

The benchmark suite comprises 13 curated scenarios (C1–C13), each targeting a specific correctness or performance property of the lock system. Scenarios are run in three modes: *single* (one model, one θ), *matrix* (three models × three θ values × 13 cases = 117 runs), and *soak* (sustained load for several minutes). Table 3 describes the 13 cases.

All runs execute inside Docker Compose on a single host. Each DSCC node runs in its own container with CPU and memory limits. The benchmark runner (`benchmark_runner.cpp`) orchestrates agent threads, collects per-operation timing, and writes JSON result files consumed by the plotting pipeline.

8.2 Agent Wait Time and Latency Decomposition

Figure 7 shows agent wait time across the 13 cases. Cases C1 and C11 exhibit the highest wait times, correctly reflecting deep queues in high-contention semantic regions. Figure 8 decomposes end-to-end latency into four components. Queue Wait dominates in contended cases; the Qdrant Write Window is stable across all cases, confirming that the vector database is not a bottleneck.

8.3 Case Studies: Thundering Herd and Queue Hopping

Figure 9 shows agent-level timelines for C1 and C8. In C1, all 13 agents arrive nearly simultaneously. Under the early single-CV design, all 12 waiting threads would have been woken by the first

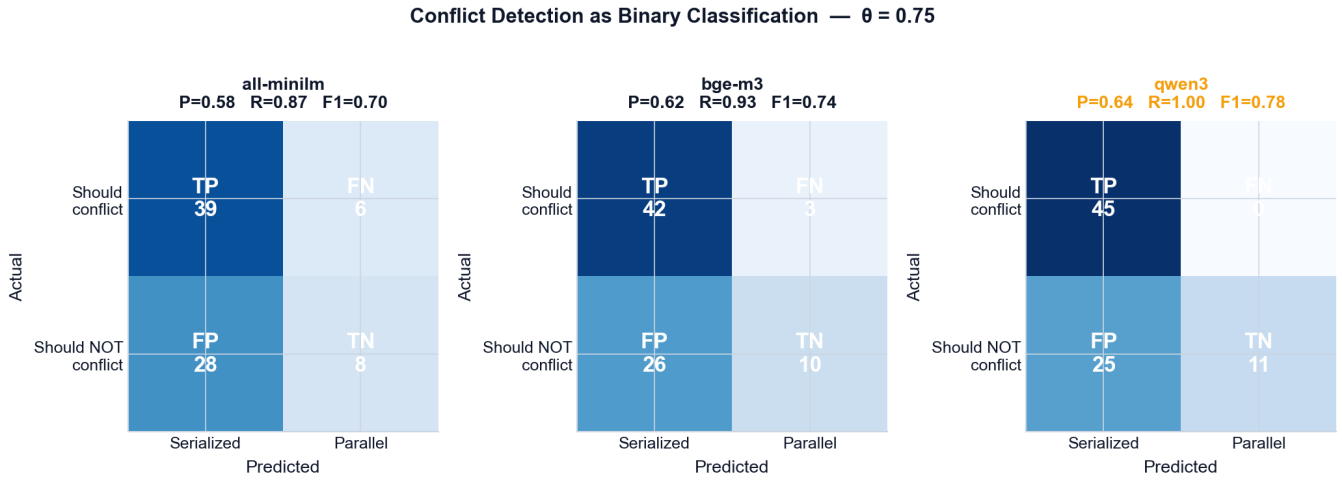


Figure 5: Per-model confusion matrices at θ=0.75. Each cell counts (conflict pair, outcome) combinations. `qwen3` has zero false negatives; `all-minilm` has 6 false negatives corresponding to the 6 paraphrase violations.

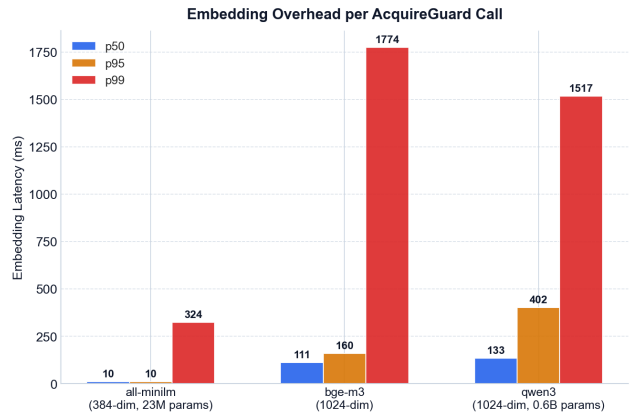


Figure 6: Embedding latency distribution for the three models. `all-minilm` is fastest (p50 ≈ 10 ms) but sacrifices semantic precision. `qwen3` has a p50 of 133 ms, which is bounded and acceptable given that the Qdrant write and lock wait dominate end-to-end latency in contended scenarios.

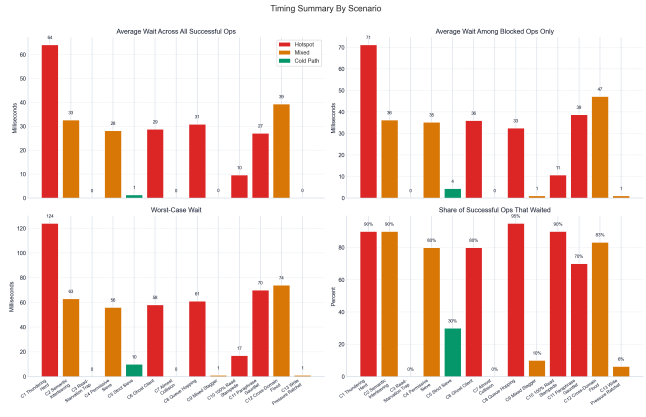


Figure 7: Agent wait time summary across all 13 benchmark cases. C1 (Thundering Herd) and C11 (Paraphrase Gauntlet) show the highest median wait times.

release — a thundering herd. Under the current per-waiter CV design, only the front-of-queue agent is woken, and the cascade proceeds one agent at a time in FIFO order. In C8, agents migrate between conflict groups as earlier waiters are granted and new pending slots appear; the timeline shows the `queue_hops` counter incrementing with each migration.

8.4 Matrix Results: Latency, Fairness, and Blocked Rate

Figure 10 shows wait fairness and blocked rate across the full matrix. At θ=0.75, the FIFO rebalancer produces near-uniform fairness across all three models: the per-waiter CV design ensures that no thread waits longer than its queue position warrants. At θ=0.55, `bge-m3` and `qwen3` block a large fraction of cross-domain requests, degrading fairness as the queue fills with operations that semantically should not conflict.

8.5 Resource Utilization

Per-container resource telemetry was collected via Docker stats snapshots at the end of each benchmark case. The leader node accounts for the majority of CPU load: it runs the embedding cosine comparison, the Raft proposal path, and the `ActiveLockTable` rebalancer on every acquire/release. Follower nodes spike briefly during `AppendEntries` log replication but are otherwise quiescent between heartbeats. Memory consumption across all nodes is dominated by the in-memory Raft log, which grows proportionally to the number of committed acquire/release operations and is bounded by the total number of benchmark operations per run.

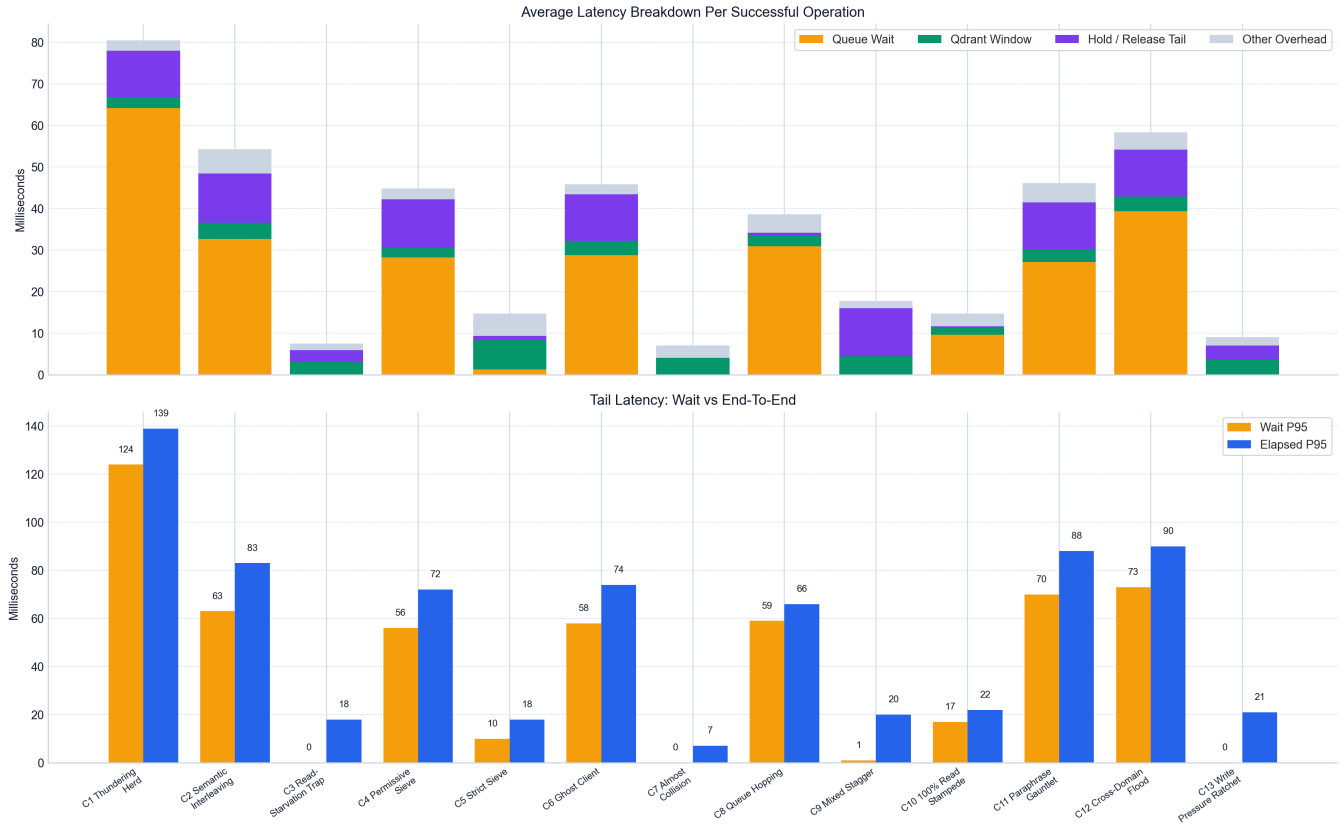


Figure 8: End-to-end latency broken down into four components: Queue Wait, Qdrant Write Window, Hold/Release Tail, and Other. In high-contention cases, Queue Wait dominates. Qdrant Write Window remains stable across cases, confirming that the vector store itself is not a bottleneck.

8.6 Locust Load Testing

We ran sustained Locust [16] load tests to characterize throughput at steady state. Two workloads were tested:

- **Read-only**: 46 RPS, median latency 7 ms, P95 13 ms.
- **Write workload**: 32 RPS, median latency 10 ms, P95 68 ms.

The write workload P95 of 68 ms includes the Raft consensus round-trip (two heartbeat periods = 150 ms upper bound), the Qdrant write, and any queue wait. The P95 being well below the heartbeat-driven upper bound indicates that the cluster processes most requests within a single AppendEntries cycle.

8.7 CAP Positioning

DSCC deliberately prioritizes Consistency over Availability. An `AcquireGuard` request to a partitioned minority cluster (fewer than 3 nodes) will timeout rather than grant a lock that might conflict with a lock held by the majority. This matches the intended use case: a semantic race condition in shared agent memory is more harmful than a temporary unavailability during a network partition.

9 LIMITATIONS AND FUTURE WORK

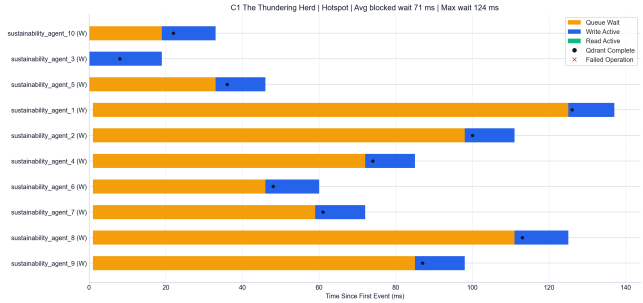
Sequential vote collection. Leader election issues `RequestVote` RPCs sequentially, giving $O(N \times \text{rpc\_timeout})$ election time in the worst case. A parallel implementation using `std::async` or co-routines would reduce this to $O(\text{rpc\_timeout})$.

No pre-vote. A node that was partitioned and incremented its term will disrupt the cluster on reconnect by triggering an unnecessary election. The pre-vote extension to Raft [4] would address this; our implementation does not include it.

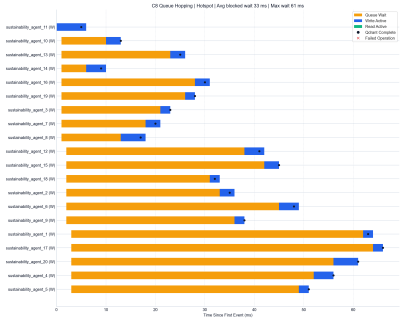
gRPC exponential backoff. A single dropped packet can push a gRPC channel into its backoff window, silencing heartbeats and triggering false elections. Channel-level keepalive tuning or application-level retry budgets would mitigate this.

No lock TTL or lease. A lock held by a crashed agent will remain in the `ActiveLockTable` indefinitely, blocking all subsequent conflicting requests. Lease-based expiry with periodic heartbeating from the agent would allow automatic cleanup.

In-memory Raft log. The current implementation does not persist the log to disk or implement log compaction via snapshots. A node that restarts loses its log state and must rejoin as a fresh follower, potentially requiring log replay from the leader.

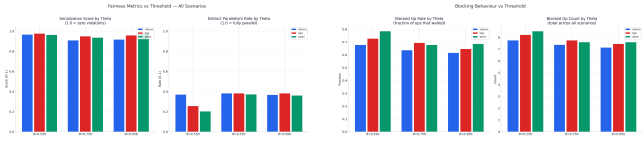


(a) C1: Thundering Herd — 13 agents serialized in FIFO order.



(b) C8: Queue Hopping — `queue_hops` increments on each migration.

Figure 9: Agent-level timing timelines for selected benchmark cases.



(a) Wait fairness vs. θ **(b) Blocked rate vs. θ**

Figure 10: Matrix benchmark results across all model-threshold combinations. Fairness and blocked rate are model-agnostic at θ=0.75; they become model-specific at extremes.

No lock ownership enforcement. Any caller can release any lock by agent ID. A token-based or challenge-response ownership check would prevent accidental or malicious releases.

Future: semantic lock federation. In large-scale deployments with many agents and diverse semantic domains, a single lock table may become a bottleneck. Partitioning the lock space by embedding cluster — effectively sharding semantic regions across multiple DSCC clusters — is an open design question.

10 CONCLUSION

We presented DSCC, a distributed lock manager that extends the classical conflict predicate from key identity to semantic similarity. The core contribution — the **ActiveLockTable** with per-lock FIFO queues and per-waiter condition variables — eliminates thundering-herd contention while preserving FIFO fairness through a cascading grant rebalancer. A two-phase admission protocol integrates the lock table with a five-node Raft cluster, and a ghost-lock prevention mechanism ensures that failed consensus proposals leave no permanent residue in the lock table.

Our embedding model evaluation demonstrates that the choice of model is an architectural commitment, not an implementation detail: the same conflict threshold can produce zero violations or six violations depending on which model maps the payload into vector space. We recommend `qwen3-embedding:0.6b` at θ=0.75 as the calibrated operating point, the only tested combination to achieve a perfect Paraphrase Gauntlet serialization score.

DSCC demonstrates that semantic concurrency control is tractable for multi-agent shared memory systems at the throughputs and latencies expected in LLM agent pipelines, opening a path toward memory-safe multi-agent architectures that do not rely on application-level deduplication or post-hoc conflict resolution.

AUTHOR CONTRIBUTIONS

Ashwattha Phatak: Designed and implemented the `AcquireGuard` critical path, the `ActiveLockTable` (including the per-waiter CV design, the rebalancer, pending lock semantics, and the cascading grant effect), the gRPC proto definitions, and the embedding model evaluation study. Designed and ran the cosine-similarity conflict predicate calibration experiments.

Ayush Gala: Designed and implemented the Raft consensus layer (`RaftNode`), the leader election and log replication subsystems, the benchmark harness (all 13 cases), node lifecycle management, the Locust load testing framework, and the leader-aware `dscc-proxy`.

ACKNOWLEDGMENTS

This work was completed as the final project for CSC 724: Advanced Topics in Distributed Systems at North Carolina State University, Spring 2026. We thank the course instructors for their feedback and for providing the computational resources used in benchmarking.

REFERENCES

[1] M. Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 335–350, 2006.

[2] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for internet-scale systems. In *Proceedings of the 2010 USENIX Annual Technical Conference (USENIX ATC)*, 2010.

[3] S. Sanfilippo. Distributed locks with Redis. https://redis.io/docs/manual/patterns/distributed-locks/, 2016.

[4] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, pages 305–320, 2014.

[5] J. N. Gray, R. A. Lorie, G. R. Putzolu, and I. L. Traiger. Granularity of locks and degrees of consistency in a shared data base. In *Modelling in Data Base Management Systems*, pages 365–394. North-Holland, 1976.

[6] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Transactions on Database Systems*, 6(2):213–226, 1981.

[7] P. A. Bernstein and N. Goodman. Multiversion concurrency control — theory and algorithms. *ACM Transactions on Database Systems*, 8(4):465–483, 1983.

[8] H. Garcia-Molina and K. Salem. Semantic concurrency control. In *Proceedings of the 1983 ACM SIGMOD International Conference on Management of Data*, pages 220–224, 1983.

[9] Qdrant Team. Qdrant: Vector similarity search engine and vector database. https://qdrant.tech, 2023.

[10] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[11] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[12] LangChain, Inc. LangGraph: Building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.

[13] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[14] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

[16] Locust Contributors. Locust: An open source load testing tool. https://locust.io, 2024.